

Program 4 – XML/JSON Parsing

Title

Devise an application to parse an XML file and display the data using appropriate UI.

Problem Description

Create an XML file with student data. On clicking a button, parse the file and display the data in a TextView.

Theory

Android provides XmlPullParser to read XML files stored in the res/xml/ folder. It works as an event-based parser: START_TAG, TEXT, and END_TAG events are fired as the parser moves through the document. This is lightweight and recommended over DOM parsers for mobile devices. The XML file is accessed via resources.xml(R.xml.data).

Code

data.xml (res/xml/data.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<students>
  <student>
    <name>John</name>
    <roll>101</roll>
    <course>Computer Science</course>
  </student>
  <student>
    <name>Mary</name>
    <roll>102</roll>
    <course>Electronics</course>
  </student>
</students>
```

activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/btnParse"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Parse XML" />

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="16sp" />

</LinearLayout>
```

MainActivity.kt

```
package com.example.xmlparserdemo

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
```

```

import org.xmlpull.v1.XmlPullParser

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnParse = findViewById<Button>(R.id.btnParse)
        val tvResult = findViewById<TextView>(R.id.tvResult)

        btnParse.setOnClickListener { parseXML(tvResult) }
    }

    private fun parseXML(tvResult: TextView) {
        val parser = resources.getXml(R.xml.data)
        val result = StringBuilder()
        var tag = ""

        while (parser.eventType != XmlPullParser.END_DOCUMENT) {
            when (parser.eventType) {
                XmlPullParser.START_TAG -> tag = parser.name
                XmlPullParser.TEXT -> {
                    val text = parser.text.trim()
                    if (text.isNotEmpty()) result.append("$tag: $text\n")
                }
            }
            parser.next()
        }
        tvResult.text = result.toString()
    }
}

```

Program 5 – SQLite Database CRUD

Title

Build a mobile application that creates, saves, updates and deletes data in a SQLite database.

Problem Description

UI with EditTexts for student ID, name and marks. Four buttons: Insert, Update, Delete, View. Data is displayed in an AlertDialog on View.

Theory

SQLite is a lightweight relational database built into Android. SQLiteOpenHelper is the helper class used to manage database creation and versioning. The onCreate() method runs the CREATE TABLE SQL when the app is first installed. ContentValues is used to bind column values for insert/update. Cursors are used to iterate over query results. All operations run on the main thread here for simplicity.

Code

activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText android:id="@+id/etId"    android:hint="Student ID"
    android:inputType="number" android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
    <EditText android:id="@+id/etName"  android:hint="Student Name"
    android:layout_width="match_parent" android:layout_height="wrap_content"/>
    <EditText android:id="@+id/etMarks" android:hint="Marks"
    android:inputType="number" android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
    <Button android:id="@+id/btnInsert" android:text="Insert"
    android:layout_width="wrap_content" android:layout_height="wrap_content"/>
    <Button android:id="@+id/btnUpdate" android:text="Update"
    android:layout_width="wrap_content" android:layout_height="wrap_content"/>
    <Button android:id="@+id/btnDelete" android:text="Delete"
    android:layout_width="wrap_content" android:layout_height="wrap_content"/>
    <Button android:id="@+id/btnView"   android:text="View Data"
    android:layout_width="wrap_content" android:layout_height="wrap_content"/>

</LinearLayout>
```

DBHelper.kt

```
import android.content.ContentValues
import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class DBHelper(context: Context) : SQLiteOpenHelper(context, "StudentDB", null, 1) {

    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL("CREATE TABLE students(id INTEGER PRIMARY KEY, name TEXT, marks
INTEGER)")
    }

    override fun onUpgrade(db: SQLiteDatabase, old: Int, new: Int) {
        db.execSQL("DROP TABLE IF EXISTS students"); onCreate(db)
    }
}
```

```

    fun insert(id: Int, name: String, marks: Int): Boolean {
        val cv = ContentValues().apply { put("id",id); put("name",name);
put("marks",marks) }
        return writableDatabase.insert("students", null, cv) != -1L
    }

    fun update(id: Int, name: String, marks: Int): Boolean {
        val cv = ContentValues().apply { put("name",name); put("marks",marks) }
        return writableDatabase.update("students", cv, "id=?", arrayOf("$id")) > 0
    }

    fun delete(id: Int): Boolean =
        writableDatabase.delete("students", "id=?", arrayOf("$id")) > 0

    fun getAll(): String {
        val cur = readableDatabase.rawQuery("SELECT * FROM students", null)
        val sb = StringBuilder()
        while (cur.moveToNext())
            sb.append("ID:${cur.getInt(0)} Name:${cur.getString(1)}
Marks:${cur.getInt(2)}\n")
        cur.close(); return sb.toString()
    }
}

```

MainActivity.kt

```

import android.os.Bundle
import android.widget.*
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    lateinit var db: DBHelper

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        db = DBHelper(this)

        val etId    = findViewById<EditText>(R.id.etId)
        val etName  = findViewById<EditText>(R.id.etName)
        val etMarks = findViewById<EditText>(R.id.etMarks)

        fun id()      = etId.text.toString().toInt()
        fun name()    = etName.text.toString()
        fun marks()   = etMarks.text.toString().toInt()
        fun toast(msg: String) = Toast.makeText(this, msg, Toast.LENGTH_SHORT).show()

        findViewById<Button>(R.id.btnInsert).setOnClickListener {
            toast(if (db.insert(id(), name(), marks())) "Inserted" else "Failed")
        }
        findViewById<Button>(R.id.btnUpdate).setOnClickListener {
            toast(if (db.update(id(), name(), marks())) "Updated" else "Failed")
        }
        findViewById<Button>(R.id.btnDelete).setOnClickListener {
            toast(if (db.delete(id())) "Deleted" else "Failed")
        }
        findViewById<Button>(R.id.btnView).setOnClickListener {
            AlertDialog.Builder(this)
                .setTitle("Student Records").setMessage(db.getAll())
                .setPositiveButton("OK", null).show()
        }
    }
}

```


Program 7 – Receive Incoming SMS & Notify User

Title

Create an application to receive an incoming SMS, notify the user, and display the message content and sender number on clicking the notification.

Problem Description

The app registers a BroadcastReceiver for incoming SMS. When a message arrives, a notification is shown via NotificationManager with a PendingIntent. Tapping the notification opens MainActivity displaying the sender number and message.

Theory

BroadcastReceiver listens for system broadcasts such as android.provider.Telephony.SMS_RECEIVED. The SMS PDU (Protocol Data Unit) is decoded using SmsMessage.createFromPdu(). A PendingIntent wraps an Intent so that the notification system can fire it later on the user's behalf. NotificationManager.notify() posts the notification. The RECEIVE_SMS permission must be declared in AndroidManifest and granted at runtime.

Code

AndroidManifest.xml (permissions + receiver)

```
<uses-permission android:name="android.permission.RECEIVE_SMS"/>
<uses-permission android:name="android.permission.READ_SMS"/>

<!-- inside <application> -->
<receiver android:name=".SmsReceiver"
    android:exported="true">
    <intent-filter android:priority="999">
        <action android:name="android.provider.Telephony.SMS_RECEIVED"/>
    </intent-filter>
</receiver>
```

SmsReceiver.kt

```
package com.example.mysms

import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.os.Build
import android.telephony.SmsMessage
import androidx.core.app.NotificationCompat

class SmsReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        val bundle = intent.extras ?: return
        val pdus = bundle["pdus"] as Array<*>

        var sender = ""; var body = ""
        for (pdu in pdus) {
            val msg = SmsMessage.createFromPdu(pdu as ByteArray)
            sender = msg.originatingAddress ?: ""
            body += msg.messageBody
        }

        // Build intent to open MainActivity with SMS data
        val openIntent = Intent(context, MainActivity::class.java).apply {
```

```

        putExtra("sender", sender)
        putExtra("body", body)
        flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
    }
    val pi = PendingIntent.getActivity(
        context, 0, openIntent, PendingIntent.FLAG_UPDATE_CURRENT or
        PendingIntent.FLAG_IMMUTABLE
    )

    // Create notification channel (required API 26+)
    val channelId = "sms_channel"
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val chan = NotificationChannel(channelId, "SMS",
            NotificationManager.IMPORTANCE_HIGH)

        context.getSystemService(NotificationManager::class.java).createNotificationChannel(chan)
    }

    val notif = NotificationCompat.Builder(context, channelId)
        .setSmallIcon(android.R.drawable.ic_dialog_email)
        .setContentTitle("SMS from $sender")
        .setContentText(body)
        .setContentIntent(pi)
        .setAutoCancel(true)
        .build()

    (context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager)
        .notify(1, notif)
}
}

```

MainActivity.kt

```

package com.example.mysms

import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val tvSender = findViewById<TextView>(R.id.tvSender)
        val tvMessage = findViewById<TextView>(R.id.tvMessage)

        val sender = intent.getStringExtra("sender") ?: "-"
        val body = intent.getStringExtra("body") ?: "-"

        tvSender.text = "From: $sender"
        tvMessage.text = "Message: $body"
    }
}

```

activity_main.xml

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical" android:padding="24dp"
    android:layout_width="match_parent" android:layout_height="match_parent">

    <TextView android:id="@+id/tvSender"
        android:textSize="18sp" android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
    <TextView android:id="@+id/tvMessage"
        android:textSize="16sp" android:paddingTop="12dp"

```

```
        android:layout_width="match_parent" android:layout_height="wrap_content"/>
    </LinearLayout>
```


Program 8 – Media Player Application

Title

Design and develop a media player application with Play, Pause, Forward, Backward controls and a SeekBar.

Problem Description

Load an audio file (song.mp3) from res/raw. Buttons control playback. A SeekBar shows progress and allows scrubbing. A background thread updates the SeekBar every second.

Theory

MediaPlayer is the core Android API for audio/video playback. It is created with MediaPlayer.create() which handles prepare() automatically for local resources. seekTo(ms) jumps to a position. A separate Thread is used to poll currentPosition and update the SeekBar asynchronously, since UI updates cannot block the main thread. SeekBar.OnSeekBarChangeListener allows the user to drag and seek. onDestroy() calls release() to free resources.

Code

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical" android:padding="16dp"
    android:layout_width="match_parent" android:layout_height="match_parent">

    <Button android:id="@+id/buttonPlay"    android:text="Play"
    android:layout_width="match_parent" android:layout_height="wrap_content"/>
    <Button android:id="@+id/buttonPause"    android:text="Pause"
    android:layout_width="match_parent" android:layout_height="wrap_content"/>
    <Button android:id="@+id/buttonForward"    android:text="Forward"
    android:layout_width="match_parent" android:layout_height="wrap_content"/>
    <Button android:id="@+id/buttonBackward"    android:text="Backward"
    android:layout_width="match_parent" android:layout_height="wrap_content"/>
    <SeekBar android:id="@+id/seekBar"
        android:layout_width="match_parent" android:layout_height="wrap_content"/>

</LinearLayout>
```

MainActivity.kt

```
package com.example.media

import android.media.MediaPlayer
import android.os.Bundle
import android.widget.Button
import android.widget.SeekBar
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    private lateinit var mediaPlayer: MediaPlayer
    private lateinit var seekBar: SeekBar

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        seekBar      = findViewById(R.id.seekBar)
        mediaPlayer = MediaPlayer.create(this, R.raw.song)
        seekBar.max = mediaPlayer.duration
    }
}
```

```

        findViewById<Button>(R.id.buttonPlay).setOnClickListener {
mediaPlayer.start() }
        findViewById<Button>(R.id.buttonPause).setOnClickListener {
mediaPlayer.pause() }
        findViewById<Button>(R.id.buttonForward).setOnClickListener {
mediaPlayer.seekTo(mediaPlayer.currentPosition + 5000) }
        findViewById<Button>(R.id.buttonBackward).setOnClickListener {
mediaPlayer.seekTo(mediaPlayer.currentPosition - 5000) }

        // Update SeekBar every second
        Thread {
            while (!isDestroyed) {
                if (mediaPlayer.isPlaying) seekBar.progress = mediaPlayer.currentPosition
                Thread.sleep(1000)
            }
        }.start()

        seekBar.setOnSeekBarChangeListener(object : SeekBar.OnSeekBarChangeListener {
            override fun onProgressChanged(sb: SeekBar?, progress: Int, fromUser:
Boolean) {
                if (fromUser) mediaPlayer.seekTo(progress)
            }
            override fun onStartTrackingTouch(sb: SeekBar?) {}
            override fun onStopTrackingTouch(sb: SeekBar?) {}
        })
    }

    override fun onDestroy() { super.onDestroy(); mediaPlayer.release() }
}

```